4]

sudo apt update----------sudo apt install apt-transport-https ca-certificates curl software-properties-common -y

sudo apt install docker.io -y-----sudo systemctl enable docker

sudo docker run hello-world---mkdir my-pythonapp

cd my-pythonapp----nano app.py----nano Dockerfile

nano requirements.txt---sudo systemctl start docker

sudo systemctl status docker

docker build -t myapp-image .

docker image/s

docker stop myapp-container---docker rm myapp-container

docker run -d -p 5000:5000 --name myapp-container myapp-image---docker ps

i) from flask import Flask---app = Flask(__name__)

@app.route("/")----def home():

return "Hello!Docker is running this Python app successfullyyyy "

if __name__ == "__main__":

app.run(host="0.0.0.0", port=5000)

ii) FROM python:3.11-slim---WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py . --------EXPOSE 5000---CMD ["python", "app.py"]

iii)flask

5] git init

git config --global user.name ""

git config --global user.email

git add .------git commit -m "initial commit"

git remote add origin

git branch -m master

git push -u origin master

sudo docker build -t my-pythonapp .

docker tag my-pythonapp:latest ___/my-pythonapp:latest

docker image/s

sudo docker run -d -p 9000:5000 --name myapp-container1 /my-pythonapp:latest

docker login -u (doc dec pass)

sudo docker push /my-pythonapp:latest

(check in dd images myhub)

New ter—sudo apt update

sudo apt install openjdk-21-jdk -y

sudo rm Jenkins.war-----sudo rm -rf ~/.jenkins

wget https://get.jenkins.io/war-stable//latest/jenkins.war

java -jar Jenkins.war –httpPort=8081

cred-----newitem

6] java –version----sudo apt update---sudo apt install maven -y

**mvn archetype:generate -DgroupId=com.example.app **

**-DartifactId=demo-app **

**-DarchetypeArtifactId=maven-archetype-quickstart **

-DinteractiveMode=false----cd demo-app-v2----nano pom.xml

nano src/main/java/com/example/app/App.java

nano src/test/java/com/example/app/AppTest.java

mvn clean install

mvn exec:java -Dexec.mainClass="com.example.app.App"

next git init

git config --global user.name ""

git config --global user.email

git add .------git commit -m "initial commit"

git remote add origin

git branch -m master---git push -u origin master

do Jenkins----- sudo apt update

sudo apt install openjdk-21-jdk -y

sudo rm Jenkins.war-----sudo rm -rf ~/.jenkins

wget https://get.jenkins.io/war-stable//latest/jenkins.war

java -jar Jenkins.war –httpPort=8081

tools—JDK21,Maven------plugins-maven inti,junit

cred---pass(token)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.example.app</groupId>
 <artifactId>demo-app</artifactId>
 <packaging>jar</packaging>
 <version>1.0-SNAPSHOT</version>
 <name>demo-app</name>
 <url>http://maven.apache.org</url>
 <dependencies>
  <dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
    <version>1.4.14</version>
  </dependency>
  <!-- JUnit for testing -->
  <dependency>
     <groupId>junit</groupId>
     <artifactId>junit</artifactId>
     <version>4.13.2</version>
     <scope>test</scope>
```

```
    </dependency>
    <!-- Logging -->
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>2.0.9</version>
    </dependency>
    <!-- Apache Commons -->
    <dependency>
      <groupId>org.apache.commons</groupId>
      <artifactId>commons-lang3</artifactId>
      <version>3.14.0</version>
    </dependency>
  </dependencies>
<build>
<plugins>
      <!-- Compiler Plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.11.0</version>
        <configuration>
          <source>17</source>
```

```
            <target>17</target>
        </configuration>
      </plugin>
      <!-- Test Plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.1.2</version>
      </plugin>
    </plugins>
</build>
</project>
package com.example.app;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.apache.commons.lang3.StringUtils;
public class App {
    private static final Logger logger =
LoggerFactory.getLogger(App.class);
    public static void main(String[] args) {
      String name = "CI/CD Pipeline";
      if (StringUtils.isNotBlank(name)) {
        String message = greet(name);
```

```
        logger.info(message);
        System.out.println(message);
    } else {
        logger.error("Name is empty!");
    }
  }
  public static String greet(String name) {
    return "Hello, " + name + "! Welcome to Maven CI/CD Demo.";
  }
}
package com.example.app;
import org.junit.Test;
import static org.junit.Assert.assertEquals;
public class AppTest {
  @Test
  public void testGreet() {
    String result = App.greet("Student");
    assertEquals("Hello, Student! Welcome to Maven CI/CD Demo.", result);
  }
}
```

```
pipeline {
 agent any
 tools {
 maven 'Maven'
 jdk 'JDK21'}
 stages {
 stage('Checkout') {
 steps {
 git branch: 'main',
 url: 'https://github.com/Padmanabha187/lab-maven.git',
 credentialsId: 'github-token'}}
 stage('Build') {
 steps {
 sh 'mvn clean compile'}}
 stage('Test') {
 steps {
 sh 'mvn test'} }
 stage('Package') {
 steps {
 sh 'mvn package'
 }
 }
 stage('Run Application') {
```

```
steps {
sh 'mvn exec:java -
Dexec.mainClass="com.example.app.App"'
}
}
}
post {
success {
emailext (
subject: "SUCCESS: ${JOB_NAME} #${BUILD_NUMBER}",
body: "Build succeeded!\nCheck: ${BUILD_URL}",
to: "padmanabha462@gmail.com"
)
}
failure {
emailext (
subject: "FAILED: ${JOB_NAME} #${BUILD_NUMBER}",
body: "Build failed!\nCheck: ${BUILD_URL}",
to: "padmanabha462@gmail.com"
)
}
}
}
```

```
pipeline {
 agent any
 environment {
 DOCKER_IMAGE = "padmanabha18/my-python-app"
 DOCKER_TAG = "latest" }
 stages {stage('Build Docker Image') {
 steps {
 sh 'docker build -t $DOCKER_IMAGE:$DOCKER_TAG .'}}
 stage('Login to DockerHub') {steps {
 withCredentials([usernamePassword(
 credentialsId: 'dockerhub-credentials',
 usernameVariable: 'USER',
 passwordVariable: 'PASS')]) {
 sh 'echo $PASS | docker login -u $USER --password-stdin' } } }
 stage('Push Image') {steps {
 sh 'docker push $DOCKER_IMAGE:$DOCKER_TAG' } }
 stage('Deploy Container') {
 steps {sh '''
 docker stop myapp-container || true
 docker rm myapp-container || true
 docker run -d -p 5000:5000 \
 --name myapp-container \
 $DOCKER_IMAGE:$DOCKER_TAG ''' } } }
```